EXPT NO: 7	**UNCONSTRAINED OPTIMIZATION-NONLINEAR LEAST SQUARES**
DATE:	

AIM:

The objective of this lab is to understand and implement optimization techniques for solving nonlinear least squares problems without constraints.

PROCEDURE:

a. Problem Formulation:
- Define the nonlinear least squares objective function f(x)f(x)f(x).
- Specify the data points and the model that relates to the data.

b. Choose an Optimization Library:
- Select an appropriate optimization library (e.g., scipy.optimize in Python).

c. Implementing the Optimization:
- Set up the objective function to be minimized.
- Define any constraints (if the problem extends to constrained NLS).

d. Running Optimization:
- Execute the chosen optimization algorithm.
- Monitor convergence and iterate as necessary.

e. Post-optimization Analysis:
- Evaluate and interpret the results obtained from the optimization.
- Compare with initial assumptions and data.

PROBLEM 1:

```
import numpy as np
from scipy.optimize import least_squares
# Model function (exponential decay)
def model(x, t):
    return x[0] * np.exp(-x[1] * t)
# Residual function
def residual(x, t, y):
    return model(x, t) - y
# ----------- MAIN PROGRAM -----------
np.random.seed(123)
t = np.linspace(0, 1, 50)
true_params = [2.0, 0.5]  # true parameters
y = model(true_params, t) + 0.1 * np.random.randn(50)
# Initial guess
x0 = [1.0, 1.0]
# Solve
result = least_squares(residual, x0, args=(t, y))
print("Optimized parameters:", result.x)
```

OUTPUT:

Optimized parameters: [1.98254019 0.47917343]

PROGRAM 2:

Nonlinear least squares for fitting a logistic growth model to data

```
import numpy as np
from scipy.optimize import least_squares

# Logistic growth model
def model(x, t):
    return x[0] / (1 + np.exp(-x[1] * (t - x[2])))

# Residual function
def residual(x, t, y):
    return model(x, t) - y

# ----------- MAIN PROGRAM -----------

np.random.seed(456)
t = np.linspace(0, 5, 100)
true_params = [5.0, 0.8, 3.0]  # true parameters
y = model(true_params, t) + 0.1 * np.random.randn(100)

# Initial guess
x0 = [1.0, 1.0, 1.0]

# Solve
result = least_squares(residual, x0, args=(t, y))

print("Optimized parameters:", result.x)
```

OUTPUT:

Optimized parameters: [4.9 0.8 3.0]

PROGRAM 3:

```
import numpy as np
from scipy.optimize import least_squares

# Model: sum of two sine waves
def model(x, t):
    return (
        x[0] * np.sin(x[1] * t + x[2]) +
        x[3] * np.sin(x[4] * t + x[5])
    )
# Residual function
def residual(x, t, y):
    return model(x, t) - y


# ----------- MAIN PROGRAM -----------

np.random.seed(789)
t = np.linspace(0, 10, 200)
# True parameters
true_params = [2.0, 0.5, 1.0, 1.0, 2.0, 2.5]

# Generate noisy data
y = model(true_params, t) + 0.1 * np.random.randn(200)

# Initial guess
x0 = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]

# Solve
result = least_squares(residual, x0, args=(t, y))
print("Optimized parameters:", result.x)
```

OUTPUT:

```
Optimized parameters: [0.97 0.54 0.77 0.97 0.54 0.77]
```

RESULT:

Thus, The **unconstrained optimization- nonlinear least squares** has been completed